

Managing Multi-Cluster Kubernetes Deployments with ArgoCD

From kubectl chaos to GitOps clarity

The Problem: Traditional CI/CD

- Manual kubectl apply to multiple clusters
- Git and clusters drift out of sync
- Hard rollbacks, risky credentials in CI

Result: Complexity, Errors, Inconsistency

The Solution: ArgoCD GitOps

- Git = Single Source of Truth
- ArgoCD auto-syncs manifests to clusters
- Easy rollbacks (just revert Git commit)
- Supports multi-cluster deployments

Result: Consistent, Auditable, Secure

Why Multi-Cluster?

There are several reasons why teams adopt multiple clusters:

- **Environment separation:** staging and prod shouldn't share the same cluster.
- **Regional redundancy:** deploy apps close to users.
- **Security/compliance:** isolating workloads across teams or tenants.

The challenge: how do you keep deployments **consistent and automated** across them?

ArgoCD as a Multi-Cluster GitOps Control Plane

By default, ArgoCD manages the cluster it runs in. But it can also manage remote clusters.

ArgoCD stores credentials for multiple clusters, and you can tell each Application resource *which cluster* to deploy to.

The model looks like this:

Git Repo (source of truth)

|

ArgoCD

/ \

Staging Prod

Cluster Cluster

Pre-requisites

- Kubernetes Cluster
- Kubectl CLI
- ArgoCD CLI

GitHub Repo

<https://github.com/chandika-s/argocd-multi-cluster-demo.git>

Step 1: Add Clusters to ArgoCD

First, install ArgoCD in one cluster.

Then, add your dev and production clusters:

```

~/Desktop/cdit/Argocd$ cat ~/.kube/config
- context:
  cluster: arn:aws:eks:us-east-1:020020254242:cluster/dev-argocd-cluster
  user: arn:aws:eks:us-east-1:020020254242:cluster/dev-argocd-cluster
  name: arn:aws:eks:us-east-1:020020254242:cluster/dev-argocd-cluster
- context:
  cluster: arn:aws:eks:ap-south-1:020020254242:cluster/prod-argocd-cluster
  user: arn:aws:eks:ap-south-1:020020254242:cluster/prod-argocd-cluster
  name: arn:aws:eks:ap-south-1:020020254242:cluster/prod-argocd-cluster
current-context: arn:aws:eks:ap-south-1:020020254242:cluster/prod-argocd-cluster
kind: Config

```

```

~/Desktop/cdit/Argocd$ argocd login localhost:8080
WARNING: server certificate had error: tls: failed to verify certificate: x509: certificate signed by unknown authority. Proceed insecurely (y/n)? y
Username: admin
Password:
'admin:login' logged in successfully
Context 'localhost:8080' updated

~/Desktop/cdit/Argocd$ argocd cluster add arn:aws:eks:us-east-1:020020254242:cluster/dev-argocd-cluster
WARNING: This will create a service account 'argocd-manager' on the cluster referenced by context 'arn:aws:eks:us-east-1:020020254242:cluster/dev-argocd-cluster' with full cluster level privileges. Do you want to continue [y/N]? y
{"level":"info","msg":"ServiceAccount 'argocd-manager' created in namespace 'kube-system'", "time":"2025-09-30T07:37:36+05:30"}
{"level":"info","msg":"ClusterRole 'argocd-manager-role' created", "time":"2025-09-30T07:37:36+05:30"}
{"level":"info","msg":"ClusterRoleBinding 'argocd-manager-role-binding' created", "time":"2025-09-30T07:37:36+05:30"}
{"level":"info","msg":"Created bearer token secret for ServiceAccount 'argocd-manager'", "time":"2025-09-30T07:37:36+05:30"}
Cluster 'https://4860181E435B443E02694E312DC7B81F.gr7.us-east-1.eks.amazonaws.com' added

~/Desktop/cdit/Argocd$ argocd cluster add arn:aws:eks:ap-south-1:020020254242:cluster/prod-argocd-cluster
WARNING: This will create a service account 'argocd-manager' on the cluster referenced by context 'arn:aws:eks:ap-south-1:020020254242:cluster/prod-argocd-cluster' with full cluster level privileges. Do you want to continue [y/N]? y
{"level":"info","msg":"ServiceAccount 'argocd-manager' created in namespace 'kube-system'", "time":"2025-09-30T07:37:45+05:30"}
{"level":"info","msg":"ClusterRole 'argocd-manager-role' created", "time":"2025-09-30T07:37:45+05:30"}
{"level":"info","msg":"ClusterRoleBinding 'argocd-manager-role-binding' created", "time":"2025-09-30T07:37:45+05:30"}
{"level":"info","msg":"Created bearer token secret for ServiceAccount 'argocd-manager'", "time":"2025-09-30T07:37:45+05:30"}
Cluster 'https://F26B27FD28B211C98E147F1B3B4635BF.gr7.ap-south-1.eks.amazonaws.com' added

```

```

~/Desktop/cdit/Argocd$ argocd cluster list

```

SERVER	PROJECT	NAME	VERSION	STATUS	MESSAGE
https://4860181E435B443E02694E312DC7B81F.gr7.us-east-1.eks.amazonaws.com		arn:aws:eks:us-east-1:020020254242:cluster/dev-argocd-cluster		Unknown	Cluster has no applications and is not being monitored.
https://F26B27FD28B211C98E147F1B3B4635BF.gr7.ap-south-1.eks.amazonaws.com		arn:aws:eks:ap-south-1:020020254242:cluster/prod-argocd-cluster		Unknown	Cluster has no applications and is not being monitored.
https://kubernetes.default.svc		in-cluster		Unknown	Cluster has no applications and is not being monitored.

This stores cluster credentials in ArgoCD's config. You can now deploy apps to them.

Step 2: Organize Manifests

Just like in single-cluster GitOps, I kept my manifests structured:

```

~/Desktop/cdit/argocd-multi-cluster-demo$ tree
.
├── argo-applications
│   ├── dev.yaml
│   ├── prod.yaml
│   └── project.yaml
├── dev
│   ├── deployment.yaml
│   └── service.yaml
└── prod
    ├── deployment.yaml
    └── service.yaml

3 directories, 7 files

```

Step 3: Create Applications for Each Cluster

Here's how I defined the **Project**:

```
argo-applications > ! project.yaml
1  apiVersion: argoproj.io/v1alpha1
2  kind: AppProject
3  metadata:
4    name: guestbook
5    namespace: argocd
6  spec:
7    description: Project for test repo with all namespaced resources allowed
8    sourceRepos:
9      - https://github.com/chandika-s/argocd-multi-cluster-demo.git
10   destinations: #Add multiple clusters
11     - namespace: guestbook-dev
12       server: https://4860181E435B443E02694E312DC7BB1F.gr7.us-east-1.eks.amazonaws.com
13     - namespace: guestbook-prod
14       server: https://F26B27FD28B211C98E147F1B3B4635BF.gr7.ap-south-1.eks.amazonaws.com
15   clusterResourceWhitelist:
16     - group: ""
17       kind: Namespace
18   namespaceResourceWhitelist:
19     - group: "*"
20       kind: "*"

```

Here's how I defined the **dev app**:

```
argo-applications > ! dev.yaml
1  apiVersion: argoproj.io/v1alpha1
2  kind: Application
3  metadata:
4    name: guestbook-dev
5    namespace: argocd
6  spec:
7    project: guestbook
8    source:
9      repoURL: 'https://github.com/chandika-s/argocd-multi-cluster-demo.git' Manifest Repo URL
10     targetRevision: main
11     path: dev # where dev k8s manifests available
12   destination:
13     server: https://4860181E435B443E02694E312DC7BB1F.gr7.us-east-1.eks.amazonaws.com # 📌 from argocd cluster list
14     namespace: guestbook-dev
15   syncPolicy:
16     automated:
17       prune: true
18       selfHeal: true
19     syncOptions:
20       - CreateNamespace=true

```

And the **production app**:

```

argo-applications > ! prod.yaml
1  apiVersion: argoproj.io/v1alpha1
2  kind: Application
3  metadata:
4    name: guestbook-prod
5    namespace: argocd
6  spec:
7    project: guestbook
8    source:
9      repoURL: 'https://github.com/chandika-s/argocd-multi-cluster-demo.git'
10     targetRevision: main # Branch for prod
11     path: prod # where prod k8s manifests available
12   destination:
13     server: https://F26B27FD28B211C98E147F1B3B4635BF.gr7.ap-south-1.eks.amazonaws.com # 🐡 from argocd cluster list
14     namespace: guestbook-prod
15   syncPolicy:
16     automated:
17       prune: true
18       selfHeal: true
19     syncOptions:
20       - CreateNamespace=true

```

Both use the same repo, but ArgoCD syncs them to different clusters.

Demo Flow

Here's what I tested:

1. Pushed a change to my deployment.yaml (Increased replicas for Prod).
2. ArgoCD synced **the prod cluster as there is a change detected in the Prod manifest path**
3. Within minutes, both environments were running perfectly in the corresponding cluster.

The screenshot shows the ArgoCD web interface at <https://localhost:8080/applications>. The interface displays two applications:

- guestbook-dev**:
 - Project: guestbook
 - Labels:
 - Status: ♥ Healthy ✔ Synced
 - Repository: https://github.com/chandika-s/argocd-mul...
 - Target Re...: main
 - Path: dev
 - Destinati...: dev-cluster
 - Namespa...: guestbook-dev
 - Created At: 09/30/2025 08:05:12 (4 minutes ago)
 - Last Sync: 09/30/2025 08:08:42 (a few seconds ago)
- guestbook-prod**:
 - Project: guestbook
 - Labels:
 - Status: ♥ Healthy ✔ Synced
 - Repository: https://github.com/chandika-s/argocd-mul...
 - Target Re...: main
 - Path: prod
 - Destinati...: prod-cluster
 - Namespa...: guestbook-prod
 - Created At: 09/30/2025 08:00:46 (9 minutes ago)
 - Last Sync: 09/30/2025 08:09:02 (a few seconds ago)

Both applications have buttons for **SYNC**, **REFRESH**, and **DELETE**.

Applications / **guestbook-dev** APPLICATION DETAILS TREE

DETAILS DIFF SYNC SYNC STATUS HISTORY AND ROLLBACK DELETE REFRESH

APP HEALTH **Healthy**

SYNC STATUS **Synced to main (06a3e0a)**
 Auto sync is enabled.
 Author: Chandika S <97661137@users.noreply.github.com>
 Comment: Create service.yaml

LAST SYNC **Sync OK to 06a3e0a**
 Succeeded a minute ago (Tue Sep 30 2025 08:08:42 GMT+0530)
 Author: Chandika S <97661137@users.noreply.github.com>
 Comment: Create service.yaml

```

graph LR
    subgraph guestbook-dev
        direction TB
        svc[guestbook-ui] -- 5 minutes --> deploy[guestbook-ui]
        deploy -- 1 minute --> img[guestbook-ui-76dbbd46ff]
        img -- 1 minute --> pod1[guestbook-ui-76dbbd46ff-fm...]
        img -- 1 minute --> pod2[guestbook-ui-76dbbd46ff-ihkb]
    end
  
```

Applications / **guestbook-prod** APPLICATION DETAILS TREE

DETAILS DIFF SYNC SYNC STATUS HISTORY AND ROLLBACK DELETE REFRESH

APP HEALTH **Healthy**

SYNC STATUS **Synced to main (06a3e0a)**
 Auto sync is enabled.
 Author: Chandika S <97661137@users.noreply.github.com>
 Comment: Create service.yaml

LAST SYNC **Sync OK to 06a3e0a**
 Succeeded a minute ago (Tue Sep 30 2025 08:09:02 GMT+0530)
 Author: Chandika S <97661137@users.noreply.github.com>
 Comment: Create service.yaml

```

graph LR
    subgraph guestbook-prod
        direction TB
        svc[guestbook-ui] -- 5 minutes --> deploy[guestbook-ui]
        deploy -- 1 minute --> img[guestbook-ui-76dbbd46ff]
        img -- 1 minute --> pod1[guestbook-ui-76dbbd46ff-9s5...]
        img -- 1 minute --> pod2[guestbook-ui-76dbbd46ff-tsl57]
    end
  
```

< Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Files

- main
- base
- dev
- overlays
- prod
 - deployment.yaml
 - service.yaml

argocd-multi-cluster-demo / prod / deployment.yaml

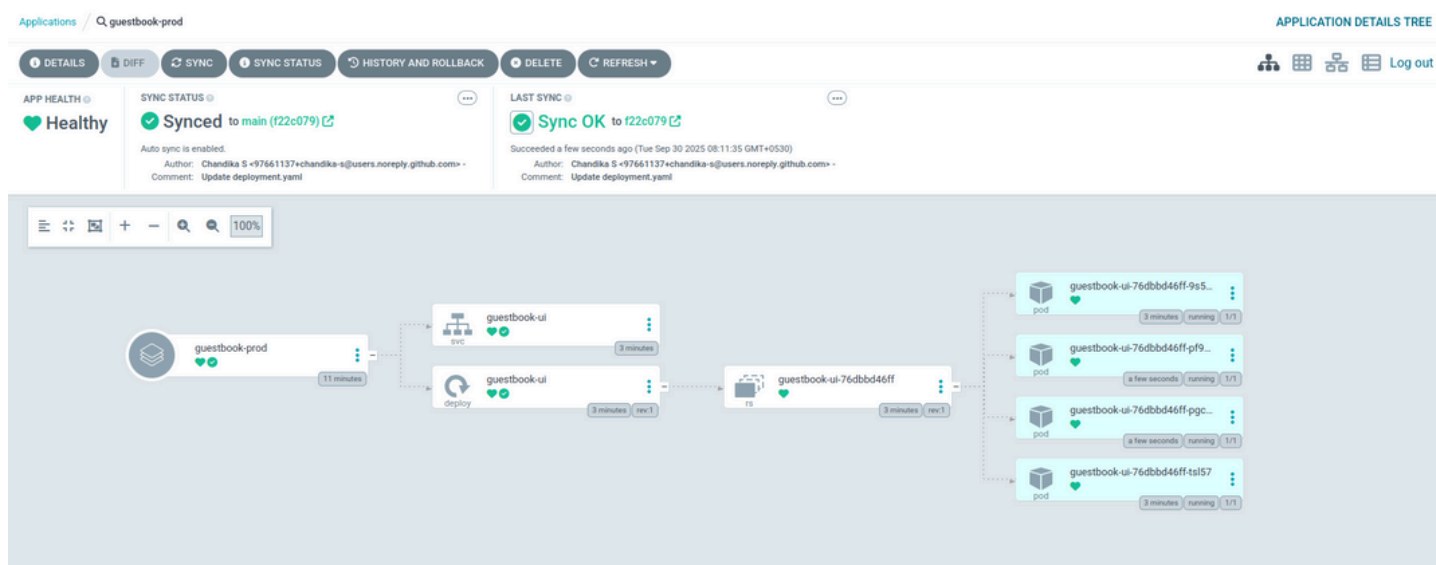
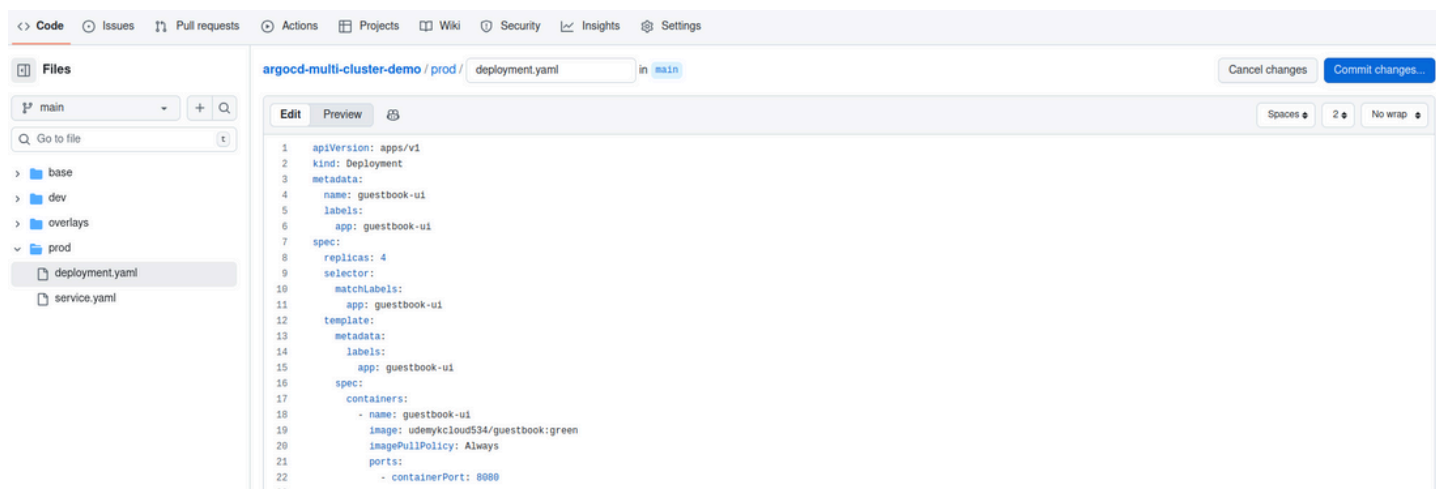
chandika-s Create deployment.yaml 9798c86 · 13 minutes ago History

Code Blame 22 lines (22 loc) · 429 Bytes

```

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: guestbook-ui
5    labels:
6      app: guestbook-ui
7  spec:
8    replicas: 2
9    selector:
10     matchLabels:
11       app: guestbook-ui
12   template:
13     metadata:
14       labels:
15         app: guestbook-ui
16     spec:
17       containers:
18         - name: guestbook-ui
19           image: udeykcld534/guestbook:green
20           imagePullPolicy: Always
21           ports:
22             - containerPort: 8080
  
```

Increased replicas for Prod **from 2 to 4**



Resources in Dev Cluster

```

~/Desktop/cdit/Argocd$ kubectl get all -n guestbook-dev
NAME                                     READY   STATUS    RESTARTS   AGE
pod/guestbook-ui-76dbbd46ff-fmq7f       1/1     Running   0           7m26s
pod/guestbook-ui-76dbbd46ff-lhlkb       1/1     Running   0           7m26s

NAME                                     TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/guestbook-ui                   ClusterIP      10.100.147.130 <none>        80/TCP      7m28s

NAME                                     READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/guestbook-ui           2/2     2             2           7m28s

NAME                                     DESIRED   CURRENT   READY   AGE
replicaset.apps/guestbook-ui-76dbbd46ff 2         2         2       7m28s

~/Desktop/cdit/Argocd$ kubectl config current-context
dev-cluster

```

Resources in Prod Cluster

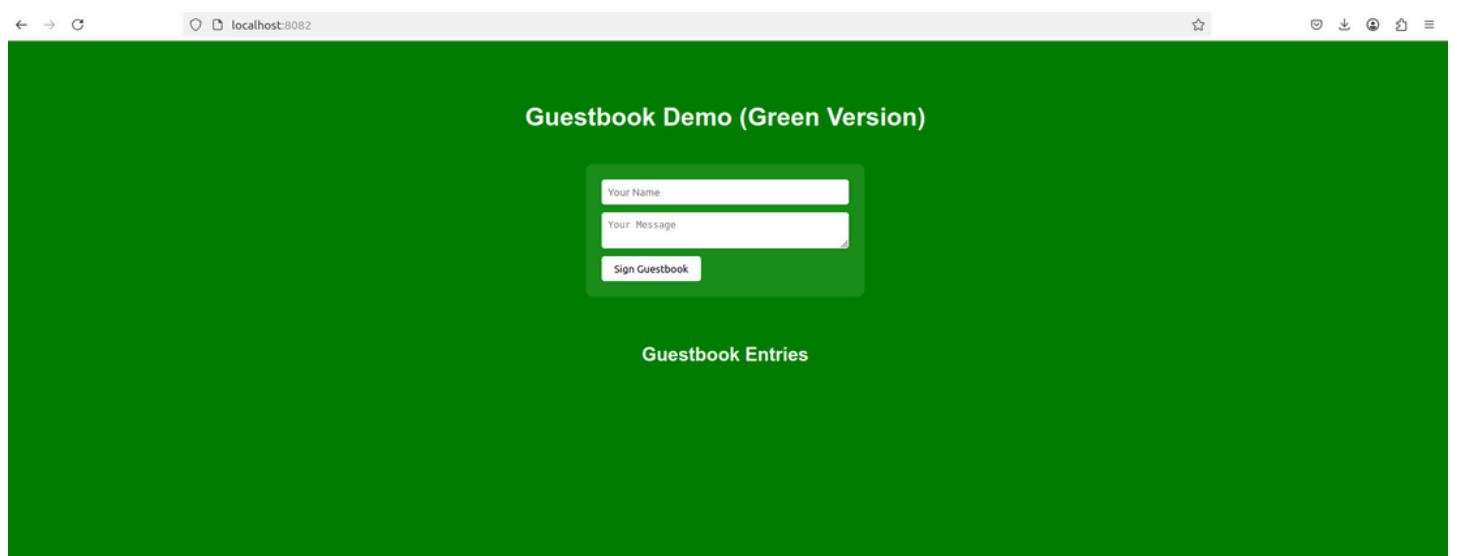

```
dev-cluster
~/Desktop/cdit/Argocd$ kubectl config current-context
dev-cluster
~/Desktop/cdit/Argocd$ kubectl config use-context prod-cluster
Switched to context "prod-cluster".
~/Desktop/cdit/Argocd$ kubectl get all -n guestbook-prod
```

NAME	READY	STATUS	RESTARTS	AGE
pod/guestbook-ui-76dbbd46ff-9s52r	1/1	Running	0	7m49s
pod/guestbook-ui-76dbbd46ff-pf95k	1/1	Running	0	5m16s
pod/guestbook-ui-76dbbd46ff-pgcgf	1/1	Running	0	5m16s
pod/guestbook-ui-76dbbd46ff-tsl57	1/1	Running	0	7m49s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/guestbook-ui	ClusterIP	10.100.228.230	<none>	80/TCP	7m49s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/guestbook-ui	4/4	4	4	7m49s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/guestbook-ui-76dbbd46ff	4	4	4	7m49s



Conclusion

Multi-cluster GitOps with ArgoCD makes deploying apps across environments **easy, consistent, and auditable**.

Instead of messy CI pipelines or manual kubectl scripts, you describe apps once in Git – and ArgoCD keeps all clusters in sync.

If your team is growing beyond a single Kubernetes cluster, I highly recommend giving ArgoCD multi-cluster support a try.